

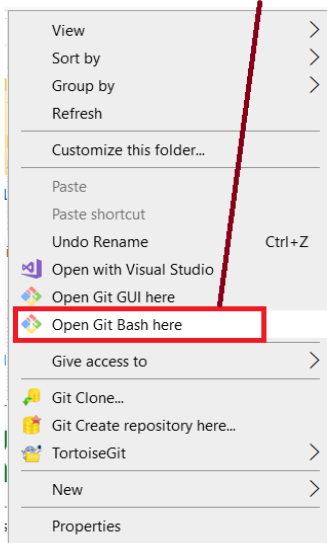
Git Commands

** Basic rule to use git commands

- 1) Every git command start with "git"
- 2) Every git command is in lower case.

To practice on git command, follow the below steps

- 1) Open your workspace folder: DevOps
- 2) Open git bash in DevOps Folder Location: Right click "Open git bash here"



- 3) Create a new directory by using below command
`mkdir git-practice`

- 4) Navigate to "git-practice" folder
`cd git-practice`

Now we can perform git commands/git command practice inside "git-practice" folder

```
MINGW64/e/DevOps/git-practice
Dell@DESKTOP-1VT9LL4 MINGW64 /e/DevOps
$ mkdir git-practice

Dell@DESKTOP-1VT9LL4 MINGW64 /e/DevOps
$ cd git-practice

Dell@DESKTOP-1VT9LL4 MINGW64 /e/DevOps/git-practice
$ pwd
/e/DevOps/git-practice
```

- 1) **git --version**: This command is used to check installed git version.

```
Dell@DESKTOP-1VT9LL4 MINGW64 /e/DevOps/git-practice
$ git --version
git version 2.46.0.windows.1
```

- 2) **git config -l** or **git config --list**

This command is used to view the git configuration information
Ones this command is executed we will get configuration information
This configuration information is available in "config" file of .git folder.

-> How to exit from "git config --list" command?

-> To exit from "git config --list" just type q. q means quite.

3) git init:

- 1) This command is used to Initialize an empty Git repository on local system or we can say git init command is used to create an empty local repository.
- 2) Every local git repository has hidden ".git" folder. This folder is automatically created when we execute "git init" command.
- 3) In real time this command is not used because we never create local repository. we clone existing repository and work on that.

```
De11@DESKTOP-1VT9LL4 MINGW64 /e/DevOps/git-practice
$ git init
Initialized empty Git repository in E:/DevOps/git-practice/.git/
```

4) git status:

git status command is used to check the status of local repository. status means state of file. In git, file can have different state like-

- 1) Untracked File
- 2) Tracked or staged Files
- 3) Committed Files
- 4) Modified Files.

1) What is mean by Untracked File?

- > The newly added file or File whose record is not available in the git is considered as Untracked File
- > In git bash, Untracked File name is displayed in red color.
- > To track the file we need to add it in the staging area by using "git add" command.

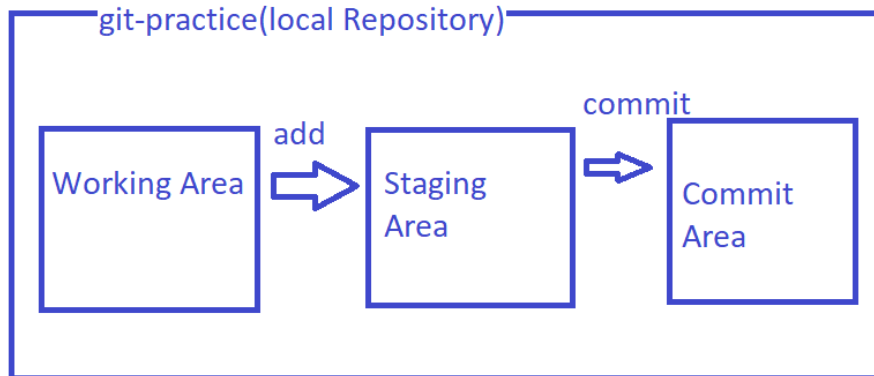
2) What is mean by Tracked File or Staged file?

- > The file present in Staging area is considered as Tracked File.
- > In git bash, Tracked File name is displayed in green color.
- > We add staged file into Commit area (local repository) by using "git commit" command

Adding untracked file/files into staging area.

-> Our Local Repository internally divided into three areas

- 1) Working area/Writing area: Newly added or modified files present
- 2) Staging area: Holds File contents Temporarily
- 3) Commit Area: File content are stored permanently



Untracked file first add into staging area using "git add" command.
Ones file is added into staging area then add it into commit area using git commit command.

git add command

1) git add command is used to add untracked files into staging area.

Case 1: Add single file into staging area.

Syntax: git add File_name

Example: git add F1.txt

Case 2: Add two or more files into staging area

Syntax: git add File1_name File2_name File3_name

Example: git add F1.txt F2.txt F3.txt

Note: File names should be separated by space.

* Case 4: Add All Files Matching a Pattern

* Syntax : git add pattern

* Example: Add all dot text files into staging
git add *.txt

* Case 3: Add all files into staging area at a time

* Syntax 1: git add --a
here --a stands for all files

* Example: git add --a

* Syntax 2: git add .

Here dot represent all files from current location

* Example: git add .

Case 5: Add a Directory into staging area

* Syntax: git add directory_name

* Example: Add "A" directory into staging
git add A

2) Adding staged files or Tracked Files into commit area (Local Repository)

- * We add staged file into local repository by using "git commit" command

- * Case 1: Add single into into commit area

- * **Syntax:** git commit File_Name -m "Commit Message"

- * **Example:** git commit F1.txt -m "Committing F1 file changes"

- * Case 2: Add two file into commit area

- * **Syntax:** git commit File_Name1 File_Name2 -m "Commit Message"

- * **Example:** git commit F1.txt F2.txt -m "Committing F1 file changes"

- * **Note:** File name should be seperated by space.

- * Case 3: Add all staged files into local repository at a time.

- * **Syntax:** git commit -m "Commit Message"

- * **Example:** git commit -m "committing all files changes"